

Automatically Translating Proof Systems for SMT Solvers to the $\lambda\Pi$ -calculus

Ciarán Dunne^{1,3} and Guillaume Burel^{2,3}

¹ INRIA, ENS Paris-Saclay;

² ensIE;

³ SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris, 91120 Palaiseau, France

Abstract. Eunoia is a logical framework designed for specifying the proofs and proof systems of SMT solvers, namely `CVC5`. We present a translation from a core fragment of Eunoia to the $\lambda\Pi$ -calculus modulo rewriting as implemented by the LambdaPi proof assistant. The translation is implemented by our tool `eo2lp`, which we use for generating LambdaPi encodings of (a) a large fragment of the *Cooperating Proof Calculus* (CPC), the Eunoia signature defining `CVC5`'s proof system, and (b) proofs produced by `CVC5` on `unsat` problems from various fragments of SMT-LIB.

1 Introduction

The area of automated reasoning known as *satisfiability modulo theories* (SMT) focuses on the development of tools for checking the satisfiability of first-order formulas over combinations of mathematical theories [9]. Such tools (known as SMT *solvers*) are increasingly used as back-ends for various tasks: hardware [27] and software verification [6, 30, 24], model checking [15], cyber-security [4, 31], and to improve automation in proof assistants [1, 10]. Because these applications demand a high level of confidence in the results produced by the solver, many solvers are capable of generating *proof certificates* that witness the unsatisfiability of formulas reported as such by the solver.

The *SMT-LIB standard* [8] specifies the syntax and semantics of a common language used for interacting with solvers. Proof generation is however not covered by the standard, leading to the development of various proof formats, namely LFSC [33], Alethe [32], and recently Eunoia [19]. Eunoia is both a proof output language and a logical framework for specifying the constants, side-condition programs, and inference rules of an SMT solver (e.g., `CVC5`). The *Cooperating Proof Calculus* (CPC) is the Eunoia signature encoding `CVC5`'s proof system, and Eunoia proof scripts may be checked by the C++ tool Ethos [19].

The $\lambda\Pi$ -calculus modulo rewriting [18] and its implementations in the tools Dedukti [3] and LambdaPi [20] provide trustworthy proof checking with an emphasis on interoperability of proof systems. This paper presents a translation from Eunoia to LambdaPi and an OCaml tool `eo2lp` that implements the translation. We used our tool to translate *CPC-MINI*, a fragment of CPC covering 14 SMT-LIB logic fragments (Fig. 1), and to further translate and typecheck over 1,000 proof scripts produced by running `CVC5` on `unsat` problems from those fragments.

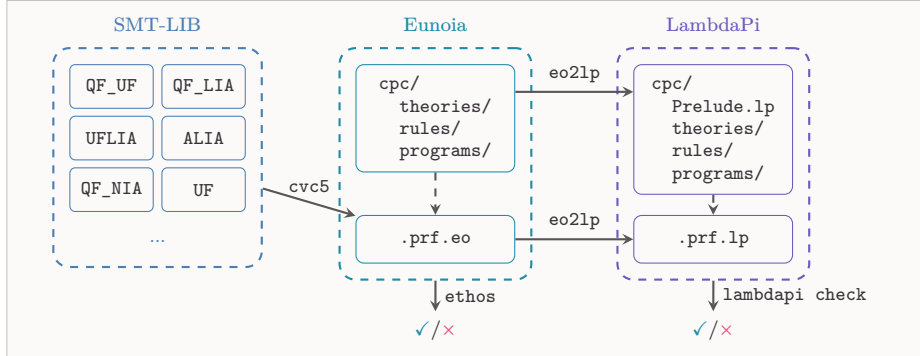


Fig. 1. Overview of the translation pipeline. CVC5 produces Eunoia proof scripts using rules from the CPC signature, which may be checked by Ethos. `eo2lp` translates both the signature and the proofs to LambdaPi, which independently verifies them.

Related Work. The reduction of SMT proof checking to type checking originates with LFSC [33]; Eunoia inherits this approach and extends it with higher-order features similar to those of the speculative SMT-LIB 3 proposal [7, 21, 5]. Brown [12] compares the prospects of interpreting the proposed type system within type-theoretic and set-theoretic frameworks; our encoding realizes the former via the $\lambda\Pi$ -calculus modulo rewriting.

The Dedukti/LambdaPi ecosystem is an interoperability hub for proof assistants (Rocq [11], Isabelle [22], HOL Light [2], Matita [25], Lean [35]) and automated provers (iProverModulo [13], ArchSAT [14], Vampire [23], TPTP [34]). Coltellacci et al. [16, 17] reconstruct `cvc5` proofs in LambdaPi via hand-written soundness lemmas for each rule of the Alethe [32] proof format. Similarly, LEAN-SMT [28] reconstructs Eunoia proofs in Lean 4 using lemmas that encode a subset of CPC’s rules. Our work instead targets Eunoia at the framework level to allow encoding arbitrary Eunoia signatures and proof scripts in a type-theoretic framework.

Outline Section 2 introduces the fragment of Eunoia targeted by our translation and its elaboration step; Section 3 defines the translation into the $\lambda\Pi$ -calculus modulo rewriting uniformly across rules, programs, and proof scripts; Section 4 describes `eo2lp` and reports the benchmark results.

2 Eunoia

This section presents the fragment of Eunoia targeted by our translation, together with the elaboration step that resolves n -ary applications to normal form. Eunoia currently has no published formal semantics: its meaning is defined operationally by Ethos. The presentation here was informed by the Ethos user manual [19], the Ethos source, and through discussions with the CVC5/Eunoia developers.

$t ::= s \mid \ell \mid (s \vec{t}) \mid (s (\vec{\rho}) t)$ (terms) $\ell ::= \text{int}(n) \mid \text{rat}(n, m) \mid \text{str}(s)$ (literals)	$\rho ::= (s t \langle \nu \rangle_?)$ (parameters) $c ::= (t t')$ (cases)
$\nu ::= \text{:implicit} \mid \text{:list}$ (prm. attributes) $\alpha ::= \text{:right-assoc} \mid \text{:right-assoc-nil } t$ (const. attributes) $\quad \mid \text{:left-assoc} \mid \text{:left-assoc-nil } t$ $\quad \mid \text{:chainable } s \mid \text{:pairwise } s$ $\quad \mid \text{:binder } s \mid \text{:arg-list } s$	
$\beta ::= \text{eo::eq} \mid \text{eo::requires} \mid \text{eo::quote} \mid \text{eo::var} \mid \dots$ (core) $\quad \mid \text{eo::and} \mid \text{eo::or} \mid \text{eo::not} \mid \text{eo::xor}$ (boolean) $\quad \mid \text{eo::add} \mid \text{eo::mul} \mid \text{eo::neg} \mid \text{eo::zdiv} \mid \dots$ (arithmetic) $\quad \mid \text{eo::nil} \mid \text{eo::cons} \mid \text{eo::list_concat} \mid \text{eo::list_len} \mid \dots$ (list)	
$\delta ::= (\text{declare-const } s t \langle \alpha \rangle_?)$ (std. commands) $\quad \mid (\text{declare-parameterized-const } s (\vec{\rho}) t \langle \alpha \rangle_?)$ $\quad \mid (\text{declare-consts } \ell t)$ $\quad \mid (\text{define } s (\vec{\rho}) t \langle \text{:type } t' \rangle_?)$ $\quad \mid (\text{program } s (\vec{\rho}) \text{:signature } (\vec{t}) t' (\vec{c}))$ $\quad \mid (\text{declare-rule } s (\vec{\rho})$ $\quad \quad \langle \text{:assumption } t_{\text{assm}} \rangle_?$ $\quad \quad \langle \text{:premises } (\vec{t}_{\text{prem}}) \mid \text{:premise-list } t_{\text{prem}} t_{\text{op}} \rangle_?$ $\quad \quad \langle \text{:args } (\vec{t}_{\text{args}}) \rangle_?$ $\quad \quad \langle \text{:requires } (\vec{c}) \rangle_?$ $\quad \quad \text{:conclusion } t_{\text{conc}})$ $\quad \mid (\text{include } \mu)$	
$\pi ::= (\text{assume } s \varphi)$ (prf. commands) $\quad \mid (\text{assume-push } s \varphi)$ $\quad \mid (\text{step } s \varphi \text{:rule } s' \langle \text{:premises } \vec{\psi} \rangle_? \langle \text{:args } \vec{t} \rangle_?)$ $\quad \mid (\text{step-pop } s \varphi \text{:rule } s' \langle \text{:premises } \vec{\psi} \rangle_? \langle \text{:args } \vec{t} \rangle_?)$	

Fig. 2. Eunoia syntax: terms, attributes, built-in symbols, commands, proof commands.

2.1 Syntax, Commands, and Signatures

Fig. 2 defines the concrete syntax for Eunoia, where the metavariable s ranges over a set of *symbols*. A term is either a symbol s , a literal ℓ (integer, rational, or string), an *application* $(s \vec{t})$, or a *binder application* $(s (\vec{\rho}) t)$. A *parameter* $(s t \langle \nu \rangle_?)$ declares a symbol s of type t with an optional *parameter attribute* ν . Similarly, a symbol may have a *constant attribute* α that specifies how its n -ary applications are elaborated to binary form. Eunoia provides a fixed set of *built-in symbols* β that operate on the syntactic structure of terms; their role is explained alongside the constructs that use them (rule declarations in Section 2.2

Table 1. Declarations produced by each Eunoia command. Parts in $\langle \cdot \rangle_?$ are present only when the corresponding attribute is supplied. Each program case $c_i = (l_i r_i)$ contributes one rewrite rule.

Command	Declarations
<code>(declare-const s t $\langle \alpha \rangle_?$)</code>	$\{s : t\} \langle \cup \{s :: \alpha\} \rangle_?$
<code>(declare-parameterized-const s $(\vec{\rho})$ t $\langle \alpha \rangle_?$)</code>	$\{s(\vec{\rho}) : t\} \langle \cup \{s :: \alpha\} \rangle_?$
<code>(declare-consts ℓ t)</code>	binds the literal class ℓ to type t
<code>(define s $(\vec{\rho})$ t $\langle \text{:type } t' \rangle_?$)</code>	$\{s(\vec{\rho}) \hookrightarrow t\} \langle \cup \{s(\vec{\rho}) : t'\} \rangle_?$
<code>(program s $(\vec{\rho})$:signature $(\vec{t})$ t' $(c_1 \dots c_m)$)</code>	$\{s(\vec{\rho}) : (\rightarrow \vec{t} t')\} \cup \{l_i \hookrightarrow r_i \mid c_i = (l_i r_i)\}$
<code>(declare-rule s $(\vec{\rho})$...)</code>	$\{s(\vec{\rho}) : \tau^*\}$ (Section 2.2)
<code>(include μ)</code>	declarations of module μ
<code>(assume s φ) / (assume-push s φ)</code>	$\{s : (\text{Proof } \varphi)\}$
<code>(step s φ :rule r $\langle \text{:args } (\vec{t}) \rangle_?$ $\langle \text{:premises } (\vec{\rho}) \rangle_?$ $\langle \text{:s } (\text{Proof } \varphi), s \hookrightarrow (r \vec{t} \vec{\rho}) \rangle_?$)</code>	

and elaboration in Section 2.3). Commands are divided into *standard commands* and *proof commands*.

A *signature* Σ is a set of *declarations*, where each declaration is either a *typing* $s(\vec{\rho}) : t$, an *attribution* $s(\vec{\rho}) :: \alpha$, or a *rewrite rule* $\ell \hookrightarrow r$ with parameters $\vec{\rho}$. Table 1 maps each command to the set of declarations it contributes. Any sequence of standard commands $\vec{\delta}$ thus determines a signature: the union of the declarations contributed by each command. A *proof script* $\Upsilon = (\vec{\delta}; \vec{\pi})$ pairs a sequence of standard commands $\vec{\delta}$ (the *preamble*) with a sequence of proof commands $\vec{\pi}$ (the *body*); Υ likewise determines a signature, which we call a *proof*.

2.2 Rule Declarations and Proofs

Two of the core built-in symbols play a role specific to rule declarations. The guard `(eo::requires  $t_1$   $t_2$   $t_3$ )` evaluates to t_3 only when t_1 and t_2 are syntactically identical, and is otherwise stuck; it lets a rule make its conclusion conditional on a side-condition. The marker `(eo::quote  $t$ )` flags t as a *quoted argument* whose value may appear in the return type of the rule, giving Eunoia's mechanism for object-level dependent types. Given an inference rule declaration `(declare-rule  $s$   $(\vec{\rho})$  ... :conclusion  $\varphi$ )`, the derivation of its contributed type declaration $s(\vec{\rho}) : \tau^*$ is given thus:

1. Given `:requires  $((l_1 r_1) \dots (l_k r_k))$` , define φ^* below. Otherwise, $\varphi^* := \varphi$.

$$\varphi^* := (\text{eo::requires } l_1 r_1 \dots (\text{eo::requires } l_k r_k \varphi))$$

2. If the declaration provides no premises, let $\tau^* := (\text{Proof } \varphi^*)$. Otherwise, define τ^* thus:

- (a) Given `:premises  $(\psi_1 \dots \psi_m)$` , let:

$$\tau^* := (\rightarrow (\text{Proof } \psi_1) \dots (\text{Proof } \psi_m) (\text{Proof } \varphi^*))$$

(b) Given `:premise-list` ψ f for some associative symbol f , redefine:

$$\varphi^\star := (\text{eo}::\text{requires } (\text{eo}::\text{is_list } f \psi) \text{ true } \varphi^\star)$$

and let $\tau^\star := (-> (\text{Proof } \psi) (\text{Proof } \varphi^\star))$.

3. If the declaration provides `:args` $(t_1 \dots t_n)$, then the type contributed for s is:

$$s(\vec{\rho}) : (-> (\text{eo}::\text{quote } t_1) \dots (\text{eo}::\text{quote } t_n) \tau^\star)$$

Otherwise, $s(\vec{\rho}) : \tau^\star$.

Example 1. Consider four rule declarations from `cpc/rules/Booleans.eo`.

```

1 (declare-rule modus_ponens ((F1 Bool) (F2 Bool))
2   :premises (F1 (=> F1 F2))
3   :conclusion F2)

1 (declare-rule cnf_implies_pos ((F1 Bool) (F2 Bool))
2   :args ((=> F1 F2))
3   :conclusion (or (not (=> F1 F2)) (not F1) F2))

1 (declare-rule reordering ((C1 Bool) (C2 Bool))
2   :premises (C1)
3   :args (C2)
4   :requires (((eo::list_mininclude or C1 C2) true))
5   :conclusion C2)

1 (declare-rule and_intro ((F Bool))
2   :premise-list F and
3   :conclusion F)

```

The `modus_ponens` rule has only `:premises` and `:conclusion`, so item 2a gives:

$$\text{modus_ponens}(\vec{\rho}) : (-> (\text{Proof } F1) (\text{Proof } (=> F1 F2)) (\text{Proof } F2))$$

The `cnf_implies_pos` rule has `:args` $((=> F1 F2))$; item 3 makes this an `eo::quote`-typed parameter that pattern-matches at the point of use, where φ is the term given by its `:conclusion` attribute:

$$\text{cnf_implies_pos}(\vec{\rho}) : (-> (\text{eo}::\text{quote } (=> F1 F2)) (\text{Proof } \varphi))$$

The `reordering` rule combines `:args` with `:requires`: item 1 builds the guarded conclusion $\varphi^\star := (\text{eo}::\text{requires } (\text{eo}::\text{list_mininclude or } C1 \ C2) \text{ true } C2)$, item 2a adds the premise $C1$, and item 3 wraps the `eo::quote`-typed argument $C2$:

$$\text{reordering}(\vec{\rho}) : (-> (\text{eo}::\text{quote } C2) (\text{Proof } C1) (\text{Proof } \varphi^\star))$$

The `and_intro` rule has `:premise-list` F and, so item 2b guards the conclusion with $\varphi^\star := (\text{eo}::\text{requires } (\text{eo}::\text{is_list and } F) \text{ true } F)$ and takes the premise list as a single `(Proof F)` argument:

$$\text{and_intro}(\vec{\rho}) : (-> (\text{Proof } F) (\text{Proof } \varphi^\star))$$

2.3 Elaboration

Unlike standard LF-style frameworks, Eunoia supports n -ary applications of binary symbols, with the unrolling determined by a *constant attribute* α (Fig. 2) assigned to the symbol at its declaration. After elaboration, every application of a declared constant s uses the built-in higher-order application operator $_$, so that $(s\ t_1\ t_2)$ becomes $(_ (_ s\ t_1)\ t_2)$; only built-in and program symbols are applied directly. For example, given f with attribute `:right-assoc-nil` t_{nil} , the f -list with elements $t_1 \dots t_n$ is the nested application $(_ (_ f\ t_1)\ (_ (_ f\ \dots)\ (_ (_ f\ t_n)\ t_{\text{nil}})))$. The built-in symbols `eo::nil`, `eo::cons`, and `eo::list_concat` construct and combine such f -lists from within programs and rule conclusions.

A second elaboration strategy targets binder applications. Eunoia provides a built-in *heterogeneous list* type `eo::List`, with constructors `eo::List::nil` and `eo::List::cons`, whose entries may have different types, and a built-in symbol `eo::var` that constructs variable-like terms $(\text{eo::var } "x" t)$. The attribute `:binder` t_{cons} specifies that a binder application $(s\ (\vec{\rho})\ t)$ is elaborated so that each parameter $(x_i\ t_i)$ in $\vec{\rho}$ becomes an `eo::var` term, and these terms are gathered with the list constructor t_{cons} (typically `eo::List::cons`, with terminator `eo::List::nil`).

The **elab** operator below captures both strategies uniformly. It is parameterized by a signature Σ and a local parameter context $\vec{\rho}$; we write $\text{progs}(\Sigma)$ for the set of symbols introduced by **program** commands in Σ .

Definition 1. For any signature Σ and parameter context $\vec{\rho}$, let $\text{elab}_{(\Sigma, \vec{\rho})}$ be the least operator such that:

1. For any symbol s , $\text{elab}_{(\Sigma, \vec{\rho})}[s] = s$.
2. For a binder application $(s\ (\vec{v})\ t)$ where $(s :: \text{:binder } t_{\text{cons}}) \in \Sigma$, each parameter $(x_i\ t_i) \in \vec{v}$ is reified as $X_i := (\text{eo::var } "x_i" t_i)$, and the X_i are gathered with the list constructor t_{cons} :

$$\text{elab}_{(\Sigma, \vec{\rho})}[(s\ (\vec{v})\ t)] = (s\ t_{\text{vars}}\ t_{\text{body}}) \quad \text{where} \quad \begin{array}{ll} t_{\text{vars}} &:= \text{elab}_{(\Sigma, \vec{\rho})}[(t_{\text{cons}}\ X_1 \dots X_n)] \\ t_{\text{body}} &:= \text{elab}_{(\Sigma, \vec{\rho})}[\text{subst}_{(\vec{x}, \vec{X})}[t]] \end{array}$$

3. For an application $(s\ \vec{t})$, let $t'_i := \text{elab}_{(\Sigma, \vec{\rho})}[t_i]$ in:
 - (a) If s is a built-in or $s \in \text{progs}(\Sigma)$, then $\text{elab}_{(\Sigma, \vec{\rho})}[(s\ \vec{t})] = (s\ \vec{t}')$.
 - (b) If $s \in \vec{\rho}$, then $\text{elab}_{(\Sigma, \vec{\rho})}[(s\ \vec{t})] = (_ \dots (_ (_ s\ t'_1)\ \dots)\ t'_n)$.
 - (c) If $(s :: \alpha) \in \Sigma$ for an associative α , the result is one of:

$$\begin{cases} \text{foldr}(G, t_{\text{nil}}, \vec{t}') & (\alpha = \text{:right-assoc-nil } t_{\text{nil}}) \\ \text{foldl}(G, t_{\text{nil}}, \vec{t}') & (\alpha = \text{:left-assoc-nil } t_{\text{nil}}) \\ \text{foldr}(G, t'_n, t'_1 \dots t'_{n-1}) & (\alpha = \text{:right-assoc}) \\ \text{foldl}(G, t'_1, t'_2 \dots t'_n) & (\alpha = \text{:left-assoc}) \end{cases}$$

where the combining function $G(x, y)$ is $(\text{eo::list_concat } s\ x\ y)$ if $(x :: \text{:list}) \in \vec{\rho}$ and $(_ (_ s\ x)\ y)$ otherwise.

- (d) If $(s :: \alpha) \in \Sigma$ for a non-associative α , then $\mathbf{elab}_{(\Sigma, \bar{\rho})}[(s \vec{t})] = \mathbf{elab}_{(\Sigma, \bar{\rho})}[(t_{\text{cons}} \vec{u})]$ where:

$$\vec{u} := \begin{cases} (s \ t_1 t_2) (s \ t_2 t_3) \dots (s \ t_{n-1} t_n) & (\alpha = \text{:chainable } t_{\text{cons}}) \\ (s \ t_i t_j) \text{ for all } i < j & (\alpha = \text{:pairwise } t_{\text{cons}}) \\ \vec{t} & (\alpha = \text{:arg-list } t_{\text{cons}}) \end{cases}$$

Example 2. The constants `or`, `=`, and `forall` carry attributes that drive each of the elaboration cases:

```

1 (declare-const or (-> Bool Bool Bool) :right-assoc-nil false)
2 (declare-parameterized-const =
3   ((A Type :implicit))
4   (-> A A Bool) :chainable and)

1 (declare-const forall (-> eo::List Bool Bool) :binder)

```

Since `or` has attribute `:right-assoc-nil false`, item 3c applies with $G(x, y) = (_ (_ \text{or } x) y)$ and the right fold gives:

$$\mathbf{elab}_{(\Sigma, \bar{\rho})}[(\text{or } x \ y \ z)] = (_ (_ \text{or } x) (_ (_ \text{or } y) (_ (_ \text{or } z) \text{false})))$$

Since `=` has attribute `:chainable` and, item 3d first chains the equalities and then recurses through `and`'s `:right-assoc-nil false` attribute:

$$\mathbf{elab}_{(\Sigma, \bar{\rho})}[(\text{= } a \ b \ c \ d)] = \mathbf{elab}_{(\Sigma, \bar{\rho})}[(\text{and } (\text{= } a \ b) (\text{= } b \ c) (\text{= } c \ d))]$$

Since `forall` has attribute `:binder eo::List::cons`, item 2 reifies each parameter as an `eo::var` term inside an `eo::List` and elaborates the body under the substitution $\sigma := (x \mapsto X)$, where $X := (\text{eo::var } "x" \text{ Int})$:

$$\begin{aligned} \mathbf{elab}_{(\Sigma, \bar{\rho})}[(\text{forall } ((x \text{ Int})) F)] &= (\text{forall } t_{\text{vars}} t_{\text{body}}) \\ t_{\text{vars}} &= (\text{eo::List::cons } X \ \text{eo::List::nil}) \\ t_{\text{body}} &= \mathbf{elab}_{(\Sigma, \bar{\rho})}[\sigma F] \end{aligned}$$

3 Encoding Eunoia in the $\lambda\Pi$ -calculus

Fig. 3 provides syntax and typing rules for the $\lambda\Pi$ -calculus *modulo* rewriting. Each term is either a *variable* x , a *constant* κ , a *universe* from $\{\text{TYPE}, \text{KIND}\}$, an *application* of two terms $(t \ t')$, or an *abstraction* $(\mathcal{B} \ x : t. t')$ where $\mathcal{B}$ is a *binder* from $\{\lambda, \Pi\}$. Terms are identified up to α -conversion (i.e., renaming of bound variables), and we assume appropriate definitions for *substitution* $(t[x \mapsto t'])$ and β -reduction $(t \rightsquigarrow_{\beta} t')$. Hereinafter, we may write $(t_1 \rightarrow t_2)$ for the term $(\Pi (x : t_1). t_2)$, where x does not occur free in t_2 .

A *typing* is an expression of the form $(t : t')$, and a *context* is a list of typings each of the form $(x : t)$. A *rewrite rule* is an expression of the form $(t \hookrightarrow t')$, where t has the form $(\kappa \ t_1 \ \dots \ t_n)$. A *signature* is a list of typings $(\kappa : t)$ (with t closed) and rewrite rules. For any signature Σ , let R_{Σ} be the smallest binary relation such that:

$\mu ::= \text{TYPE} \mid \text{KIND}$		(universes)
$t ::= x \mid \kappa \mid \mu \mid (tt') \mid (\lambda x : t. t') \mid (\Pi x : t. t')$		(terms)
(WFO)	$\frac{}{\text{wf } \emptyset}$	(WF+)
		$\frac{\Gamma \vdash_{\Sigma} t : \mu}{\text{wf } (\Gamma, (x : t))} \quad x \notin \text{dom}(\Gamma)$
(VAR)	$\frac{\text{wf } \Gamma}{\Gamma \vdash_{\Sigma} x : t} \quad (x : t) \in \Gamma$	(CON)
		$\frac{\text{wf } \Gamma \quad \Gamma \vdash_{\Sigma} t : \mu}{\Gamma \vdash_{\Sigma} \kappa : t} \quad (\kappa : t) \in \Sigma$
(UNIV)	$\frac{\text{wf } \Gamma}{\Gamma \vdash_{\Sigma} \text{TYPE} : \text{KIND}}$	(PROD)
		$\frac{\Gamma \vdash_{\Sigma} t : \text{TYPE} \quad \Gamma, (x : t) \vdash_{\Sigma} t' : \mu'}{\Gamma \vdash_{\Sigma} (\Pi x : t. t') : \mu'}$
(FUN)	$\frac{\Gamma, (x : t) \vdash_{\Sigma} e : t' \quad \Gamma \vdash_{\Sigma} (\Pi x : t. t') : \mu}{\Gamma \vdash_{\Sigma} (\lambda x : t. e) : (\Pi x : t. t')}$	
(APP)	$\frac{\Gamma \vdash_{\Sigma} e : (\Pi x : t. t') \quad \Gamma \vdash_{\Sigma} e' : t}{\Gamma \vdash_{\Sigma} (ee') : t'[x \mapsto e']}$	
(CONV)	$\frac{\Gamma \vdash_{\Sigma} e : t \quad \Gamma \vdash_{\Sigma} t' : \mu}{\Gamma \vdash_{\Sigma} e : t'} \quad (t \equiv_{\beta\Sigma} t')$	

Fig. 3. Syntax and typing rules for the $\lambda\Pi$ -calculus modulo rewriting.

1. for any rule $(\ell \hookrightarrow r) \in \Sigma$ and substitution σ , $(\ell\sigma, r\sigma) \in R_{\Sigma}$, and
2. R_{Σ} is *congruent* under application and abstraction.

Then, *equality modulo rewriting* ($\equiv_{\beta\Sigma}$) is defined as the least equivalence relation containing R_{Σ} and the β -reduction relation. The rules in Fig. 3 provide a (mutually inductive) definition of a *well-formedness* relation (wf_{Σ}) on contexts and a *typing relation* ($\vdash_{\Sigma}$) between contexts and typings, where $(\Gamma \vdash_{\Sigma} e : t)$ may be read as “ e has type t with respect to signature Σ and context Γ ”.

3.1 Translating Eunoia Terms

The typing rules of the $\lambda\Pi$ -calculus disallow abstractions on types (i.e., there are no expressions of type $\text{TYPE} \rightarrow t$ in any context). Eunoia’s types are therefore encoded as $\lambda\Pi$ -terms of type Set , using the Tarski-style universe of LambdaPi’s standard library: an injective symbol $\tau : \text{Set} \rightarrow \text{TYPE}$ lifts a Set -term to a $\lambda\Pi$ -type, and the (infix) function-space symbol $(\rightsquigarrow) : \text{Set} \rightarrow \text{Set} \rightarrow \text{Set}$ satisfies $\tau(A \rightsquigarrow B) \hookrightarrow \tau A \rightarrow \tau B$. The dependent types arising from `eo::quote` arguments are not expressible with $(\rightsquigarrow)$ alone, so we additionally introduce a dependent function space $(\rightsquigarrow^d) : \Pi T : \text{Set}. (\tau T \rightarrow \text{Set}) \rightarrow \text{Set}$ with the rule $\tau(T \rightsquigarrow^d F) \hookrightarrow \Pi x : \tau T. \tau(Fx)$; the non-dependent case is recovered by taking $F := \lambda _ . B$. We keep $(\rightsquigarrow)$ as a first-class operator rather than defining it from $(\rightsquigarrow^d)$ because

programs need a matchable function-space head: pattern variables cannot bind under a Π , but they can under $(\leadsto)$.

The translation $\llbracket \cdot \rrbracket_{\text{tm}}$ on Eunoia terms below produces $\lambda\Pi$ -terms in the abstract calculus of Fig. 3 and assumes all terms have already been processed by the **elab** operator of Section 2.3. Because some Eunoia symbols are not valid LambdaPi identifiers, the translation on symbols uses LambdaPi syntax for escaping: $\bar{s} := \llbracket s \rrbracket$ iff s contains forbidden characters, and $\bar{s} := s$ otherwise.

Definition 2. Let $\llbracket \cdot \rrbracket_{\text{tm}}$ be the least operator such that:

1. For any symbol s or literal ℓ , the following hold:

$$\llbracket s \rrbracket_{\text{tm}} = \begin{cases} \text{eo.Type} & \text{if } s = \text{Type}, \\ \bar{s} & \text{otherwise.} \end{cases} \quad \llbracket \ell \rrbracket_{\text{tm}} = \begin{cases} \text{of_Z } \llbracket n \rrbracket_{\text{tm}} & \text{if } \ell = \text{int}(n), \\ \text{of_Q } \llbracket n \rrbracket_{\text{tm}} \llbracket m \rrbracket_{\text{tm}} & \text{if } \ell = \text{rat}(n, m), \\ \text{"s"} & \text{if } \ell = \text{str}(s). \end{cases}$$

The symbols `of_Z` and `of_Q` are defined in our prelude (Section 4) and embed the integer and rational literals of LambdaPi's standard library into the carrier type of Eunoia's arithmetic sorts.

2. For arrow types, the following hold, where $(x \ T) \in \vec{\rho}$:

$$\begin{aligned} \llbracket (-> (\text{eo::quote } x) \ \vec{t}) \rrbracket_{\text{tm}} &= \llbracket T \rrbracket_{\text{tm}} \leadsto^d \lambda x. \llbracket (-> \vec{t}) \rrbracket_{\text{tm}} \\ \llbracket (-> t' \ \vec{t}) \rrbracket_{\text{tm}} &= \llbracket t' \rrbracket_{\text{tm}} \leadsto \llbracket (-> \vec{t}) \rrbracket_{\text{tm}} \\ \llbracket (-> t) \rrbracket_{\text{tm}} &= \llbracket t \rrbracket_{\text{tm}} \end{aligned}$$

3. For binary and n -ary applications, the following hold:

$$\begin{aligned} \llbracket (_ \ t_1 \ t_2) \rrbracket_{\text{tm}} &= \text{eo.app } \llbracket t_1 \rrbracket_{\text{tm}} \llbracket t_2 \rrbracket_{\text{tm}} \\ \llbracket (\text{Proof } t) \rrbracket_{\text{tm}} &= \text{eo.Proof } \llbracket t \rrbracket_{\text{tm}} \\ \llbracket (s \ \vec{t}) \rrbracket_{\text{tm}} &= \bar{s} \llbracket t_1 \rrbracket_{\text{tm}} \dots \llbracket t_n \rrbracket_{\text{tm}} \end{aligned}$$

4. After elaboration, every other binder application is unfolded by item 2 of Definition 1; only `eo::define` survives because it is a let-binder rather than a binding form. It is translated using LambdaPi's standard `let` binder:

$$\llbracket (\text{eo::define } (\vec{v}) \ t') \rrbracket_{\text{tm}} = (\text{let } \vec{w} \text{ in } \llbracket t' \rrbracket_{\text{tm}})$$

where $w_i = (\bar{x}_i := \llbracket t_i \rrbracket_{\text{tm}})$ for each $v_i = (x_i \ t_i)$.

3.2 The LambdaPi Proof Assistant

Fig. 4 defines the fragment of LambdaPi required for encoding Eunoia. It is sugar over the calculus of Fig. 3: implicit binders $[x : t]$ and placeholders $_$ are elaborated to Π -binders and metavariables during type inference, and each declaration extends Σ .

A *symbol declaration* $\langle m \rangle_? \text{ symbol } s \ \vec{\rho} : t \langle := t' \rangle_?$ introduces s with the typing $(@s : \Pi \vec{\rho}. t)$, where parameters $\vec{\rho}$ are either *explicit* $(x : t)$ or *implicit* $[x : t]$.

$m ::= \text{constant} \mid \text{sequential}$	(modifiers)	$\theta ::= \$x \mid s \langle \theta \rangle_*$	(patterns)
$\rho ::= (s : t) \mid [s : t]$	(parameters)	$r ::= (s \langle \theta \rangle_* \hookrightarrow \theta')$	(rewrite rules)
$ \begin{array}{l} c ::= \langle m \rangle_? \text{symbol } s \langle \rho \rangle_* : t \langle := t' \rangle_?; \quad (\text{commands}) \\ \quad \text{rule } r \langle \text{with } r' \rangle_*; \\ \quad \text{assert } \vdash t : t'; \\ \quad \text{require open } \langle \mu \rangle_+; \end{array} $			

Fig. 4. Syntax for a fragment of the LambdaPi proof assistant.

We write $@s$ (with an @) for the *fully-explicit* form; an unprefixed s inserts a placeholder for each leading implicit, to be filled by higher-order unification. The optional **constant** modifier forbids rewrite rules with s at the head; **sequential** matches rewrite rules with head s in their textual order; and a definition $(:= t')$ adds the rewrite rule $(s \hookrightarrow \lambda \vec{\rho}. t')$. A *rewrite rule declaration* **rule** $\theta \hookrightarrow \theta'$ adds the rule $(\theta \hookrightarrow \theta')$ to the signature, where θ and θ' are *patterns*—either a *pattern variable* $\$x$ (written $\$x$) or a *pattern application* $(s \langle \theta \rangle_*)$. The rule is valid when θ is a pattern application and every pattern variable in θ' also occurs in θ . LambdaPi additionally verifies *subject reduction* at declaration time: for every typed substitution of the pattern variables, both sides must have the same type. An *assertion* **assert** $\vdash t : t'$ checks $t : t'$ without extending the signature.

3.3 Translating Eunoia Commands and Proofs

We extend the term translation $\llbracket \cdot \rrbracket_{\text{tm}}$ (Definition 2) to parameter contexts and to both standard and proof commands. Table 2 summarises the full Eunoia-to-LambdaPi encoding; the remainder of this section formalises the operators on contexts (Definition 3), standard commands (Definition 4), and proof commands (Definition 5).

Translating Commands. Parameter contexts and two auxiliary notations support the command translation that follows.

Definition 3. Given $\vec{\rho} = (x_1 \ t_1 \ \langle \nu_1 \rangle_?) \dots (x_n \ t_n \ \langle \nu_n \rangle_?)$, define:

$$\llbracket \vec{\rho} \rrbracket_{\text{ctx}} := \rho_1^* \dots \rho_n^* \quad \text{where} \quad \rho_i^* = \begin{cases} [\bar{x}_i : \tau \llbracket t_i \rrbracket_{\text{tm}}] & \text{if } \nu_i = \text{implicit}, \\ (\bar{x}_i : \tau \llbracket t_i \rrbracket_{\text{tm}}) & \text{otherwise.} \end{cases}$$

We further write $\llbracket t \rrbracket_{\text{tm}}^\$$ for the result of applying $\llbracket \cdot \rrbracket_{\text{tm}}$ to t and replacing each free variable x with the pattern variable $\$x$; and $\text{impl}(\vec{\rho}, t)$ for the sublist of $\llbracket \vec{\rho} \rrbracket_{\text{ctx}}$ containing each $x_i \in \vec{\rho}$ that occurs free in t , marked as implicit.

Definition 4. Let δ be a standard Eunoia command. Then $\llbracket \delta \rrbracket_{\text{cmd}}$ is the list of LambdaPi commands described as follows:

Table 2. Eunoia-to-LambdaPi encoding. Symbols prefixed with `eo.` are defined in our hand-written prelude (Section 4); $\leadsto$ and $\leadsto^d$ are the function-space symbols introduced in Section 3.

Eunoia feature	LambdaPi encoding
Type universe <code>Type</code>	term $T : \text{Set}$, carrier τT
Function type $(\rightarrow A B)$	$A \leadsto B$, with $\tau(A \leadsto B) \hookrightarrow \tau A \rightarrow \tau B$
Dependent type from <code>(eo::quote x)</code>	$A \leadsto^d \lambda x. B$ (matchable head, unlike Π)
Parameter attribute <code>:implicit</code>	implicit binder $[x : \tau T]$
Macro <code>define</code>	<code>symbol</code> $s \vec{\rho} := t$
Higher-order application <code>(_ t₁ t₂)</code>	<code>eo.app</code> $t_1 t_2$
n -ary $(f \vec{t})$ with attribute	nested binary application; nil terminator via rewrite rule
Binder variables <code>(eo::var)</code>	reified into the list type <code>eo.List</code>
Program <code>(program)</code>	<code>sequential symbol</code> + one rewrite rule per case (SR-checked)
Inference rule <code>(declare-rule)</code>	<code>constant symbol</code> + auxiliary symbol with quote-pattern rewrite when <code>:args</code> present
Built-in arithmetic	prelude rules delegating to $\mathbb{Z}, \mathbb{Q}$
Conclusion <code>(eo::requires l r ψ)</code>	guard ψ applies only when $l \equiv r$ syntactically
Proof command <code>assume</code>	<code>constant symbol</code> of type <code>eo.Proof φ</code>
Proof command <code>step</code>	defined symbol + <code>assert</code>

1. If δ is a constant declaration, $\delta \vdash s(\vec{\rho}) : t$, the head of $\llbracket \delta \rrbracket_{\text{cmd}}$ is `symbol` $\bar{s} \llbracket \vec{\rho} \rrbracket_{\text{ctx}} : \tau \llbracket t \rrbracket_{\text{tm}}$. If $\delta \vdash s :: \alpha$ and α is `:right-assoc-nil` t_{nil} or `:left-assoc-nil` t_{nil} , the following rewrite rule is also produced, where `eo.nil` returns the nil-terminator of an associative symbol and `eo.as` performs type ascription:

$$\text{rule } (\text{eo.nil } \bar{s} \$T) \hookrightarrow (\text{eo.as } \llbracket t_{\text{nil}} \rrbracket_{\text{tm}} \$T)$$

2. If δ is a macro definition, $\delta \vdash s(\vec{\rho}) := t$, then $\llbracket \delta \rrbracket_{\text{cmd}}$ is `symbol` $\bar{s} \llbracket \vec{\rho} \rrbracket_{\text{ctx}} : \tau \llbracket t' \rrbracket_{\text{tm}} := \llbracket t \rrbracket_{\text{tm}}$ (with the typing omitted if no t' is supplied).
3. If δ is a program declaration with $\delta \vdash s(\vec{\rho}) : (\rightarrow t_1 \dots t_n t')$ and cases $(l_1 r_1) \dots (l_m r_m)$, then $\llbracket \delta \rrbracket_{\text{cmd}}$ declares $\bar{s}$, where $T := \tau \llbracket (\rightarrow t_1 \dots t_n t') \rrbracket_{\text{tm}}$:

```

sequential symbol  $\bar{s} \vec{\rho}^* : T$ ;
rule  $\llbracket l_1 \rrbracket_{\text{tm}}^{\$} \hookrightarrow \llbracket r_1 \rrbracket_{\text{tm}}^{\$}$ 
...
with  $\llbracket l_m \rrbracket_{\text{tm}}^{\$} \hookrightarrow \llbracket r_m \rrbracket_{\text{tm}}^{\$}$  ;

```

where $\vec{\rho}^* := \text{impl}(\vec{\rho}, T)$ binds all of the free parameters in T as implicits.

4. If δ is a rule declaration with `:args` $(t_1 \dots t_n)$ of type $\tau_1 \dots \tau_n$, then $\delta \vdash s(\vec{\rho}) : (\rightarrow (\text{eo::quote } t_1) \dots (\text{eo::quote } t_n) \tau^*)$, where τ^* encodes the premises and conclusion (Section 2.1). An auxiliary symbol s^* eliminates the `eo.quote` types by rewriting to $\llbracket \tau^* \rrbracket_{\text{tm}}$ on terms matching the argument patterns; s

then takes the arguments explicitly and uses $s^\star$ for its return type:

```

symbol  $\bar{s}^\star \bar{\sigma} : \llbracket \tau_1 \rrbracket_{\text{tm}} \rightarrow \dots \rightarrow \llbracket \tau_n \rrbracket_{\text{tm}} \rightarrow \text{TYPE};
rule  $\bar{s}^\star \llbracket t_1 \rrbracket_{\text{tm}}^\$ \dots \llbracket t_n \rrbracket_{\text{tm}}^\$ \hookrightarrow \llbracket \tau^\star \rrbracket_{\text{tm}}^\$ ;
constant symbol  $\bar{s} (\alpha_1 : \llbracket \tau_1 \rrbracket_{\text{tm}}) \dots (\alpha_n : \llbracket \tau_n \rrbracket_{\text{tm}}) : (\bar{s}^\star \alpha_1 \dots \alpha_n)$$$ 
```

Example 3. We illustrate $\llbracket \delta \rrbracket_{\text{cmd}}$ on the four declarations from [Example 1](#):

```

1 constant symbol = [A : Set] :  $\tau (A \hookrightarrow (A \hookrightarrow \text{Bool}))$ ;
2 constant symbol or :  $\tau (\text{Bool} \hookrightarrow (\text{Bool} \hookrightarrow \text{Bool}))$ ;
3 rule eo.nil or $T  $\hookrightarrow$  eo.as false $T;

1 constant symbol modus_ponens [F1 F2 :  $\tau \text{Bool}$ ]
2   : eo.Proof F1
3    $\rightarrow$  eo.Proof (eo.app (eo.app  $\Rightarrow$  F1) F2)
4    $\rightarrow$  eo.Proof F2;

1 symbol cnf_implies_pos_aux :  $\tau \text{Bool} \rightarrow \text{TYPE}$ ;
2 rule @cnf_implies_pos_aux
3   (eo.app (eo.app  $\Rightarrow$  $F1) $F2)
4    $\hookrightarrow$  eo.Proof
5     (eo.app (eo.app or (eo.app not (eo.app (eo.app  $\Rightarrow$  $F1) $F2)))
6       (eo.app (eo.app or (eo.app not $F1)) (eo.app (eo.app or $F2) false)));
7 constant symbol cnf_implies_pos
8   (x1 :  $\tau \text{Bool}$ ) : cnf_implies_pos_aux x1;

```

The polymorphic equality = and the associative or translate by case 1, the latter also producing a nil-terminator rewrite rule. modus_ponens has no `:args`, so case 1 yields its derived type directly. cnf_implies_pos has the compound argument `:args( $\Rightarrow$  F1 F2)`, triggering case 4: an auxiliary symbol matches the argument pattern and rewrites to the translated conclusion under `eo.Proof`.

Translating Proofs.

Definition 5. Let π be a proof command. Then $\llbracket \pi \rrbracket_{\text{cmd}}$ is the list of LambdaPi commands such that:

1. If π is an assumption, then $\pi \vdash s : (\text{Proof } \varphi)$:

```
constant symbol  $\bar{s} : \text{eo.Proof } \llbracket \varphi \rrbracket_{\text{tm}} ;$ 
```

2. If π is a step, then $\pi \vdash s : (\text{Proof } \varphi)$ and $\pi \vdash s := (r \ t_1 \dots t_m \ p_1 \dots p_n)$:

```

symbol  $\bar{s} := \bar{r} \ \llbracket t_1 \rrbracket_{\text{tm}} \dots \llbracket t_m \rrbracket_{\text{tm}} \ \bar{p}_1 \dots \bar{p}_n ;
assert \vdash \bar{s} : \text{eo.Proof } \llbracket \varphi \rrbracket_{\text{tm}} ;$ 
```

The body applies rule $\bar{r}$ to its arguments then premises. The type is checked separately via `assert`, which uses LambdaPi's type inference to fill implicit arguments.

Example 4. A QF_UF proof script (top) deriving `false` from `(= x0 x0)` via `eq_refl` and `eq_resolve` (CPC uses both `-` and `_` in rule names), and its LambdaPi translation (bottom) generated by Definition 4 and Definition 5:

```

1 (declare-const U Type)
2 (declare-const x0 U)
3 (define @t1 () (= x0 x0))
4 (define @t2 () (not @t1))
5 (assume @p1 @t2)
6 (step @p2 (= @t1 true) :rule eq_refl :args (x0))
7 ...
8 (step @p6 false :rule eq_resolve :premises (@p1 @p5))

1 require open cpc.Cpc;
2 constant symbol U : Set;
3 constant symbol x0 :  $\tau$  U;
4 symbol {|@t1|} := eo.app (eo.app = x0) x0;
5 symbol {|@t2|} := eo.app not {|@t1|};
6 constant symbol {|@p1|} : eo.Proof {|@t2|};
7 symbol {|@p2|} := eq_refl (x0);
8 assert  $\vdash$  {|@p2|} : eo.Proof (eo.app (eo.app = {|@t1|}) true);
9 ...
10 symbol {|@p6|} := eq_resolve ({|@p1|}) ({|@p5|});
11 assert  $\vdash$  {|@p6|} : eo.Proof false;

```

The preamble’s `U`, `x0` translate by case 1 and the `defines` by case 2 of Definition 4; the assumption `@p1` becomes a `constant` symbol of type `eo.Proof` $\llbracket \varphi \rrbracket_{\text{tm}}$ (case 1 of Definition 5); each step applies its rule to arguments then premises, with `assert` verifying the conclusion via type inference.

4 Implementation and Results

Our tool `eo2lp`⁴ is an OCaml program ($\sim 5,500$ lines) that implements Section 3, with a Menhir-generated parser [29] and a back-end built against the LambdaPi OCaml API. A hand-written `Prelude.lp` module (~ 335 lines) bootstraps the type embedding: it declares `Set`, τ , the function space $\rightarrow$, the higher-order application symbol `eo.app`, Boolean logic, and arithmetic builtins as rewrite rules over LambdaPi’s integers/rationals (e.g. `eo::add (of_Z i) (of_Z j)  $\hookrightarrow$  of_Z (i + j)`). The prelude also declares the built-in heterogeneous list type `eo.List : Set` (the encoding of Eunoia’s `eo::List`), together with its constructors `eo.List::nil` and `eo.List::cons`. These hold the `(eo::var "xi" Ti)` entries produced by `elab` on binder applications (Section 2.3, item 2). Closed expressions reduce concretely, so CPC’s `$run_evaluate` program—which recursively delegates to the `eo::add`, `eo::mul`, `eo::and`, `eo::not`, ... builtins—normalises ground terms like `(+ 2 3)` to 5.

⁴ <https://github.com/deducteam/eo2lp>

4.1 CPC-MINI

The Cooperating Proof Calculus (CPC) is the Eunoia signature that formalizes CVC5’s proof system. The full CPC comprises 35 core modules ($\sim 11,000$ lines of Eunoia) plus 16 expert modules for additional theories (separation logic, bags, finite fields, floating points, transcendentals). To validate our approach on a manageable portion, we define *CPC-MINI*,⁵ a fork of the upstream CPC included as a git submodule of `eo21p`. It is a modified subset of 22 core modules ($\sim 5,000$ lines) covering logic, integer arithmetic, arrays, sets, quantifiers, and uninterpreted functions — 6 theory modules (57 constants), 8 rule modules (353 rules, 50 programs), and 7 program modules (38 programs, 36 macros), plus a root module `Cpc` that adds 6 further rules (e.g. `evaluate`, `trust`) for 359 rules in total.

CPC-MINI applies three classes of change to the upstream CPC. (i) *Theory pruning*: the expert modules and unsupported core theories (bitvectors, strings, datatypes, reals) are removed, since their primitives are not yet in our prelude. (ii) *Type-parameter tightening*: a few CPC declarations leave their numeric parameter type unconstrained so that the same rule applies at both `Int` and `Real`; we instantiate these to `Int` where required, which suffices for our integer-arithmetic benchmarks. (iii) *Bug fixes*: several upstream typos and mismatched type parameters are patched; we intend to contribute these upstream. For instance, the upstream `$mk_cong_rhs` program in `Uf.eo` typed its arguments with the codomain rather than the domain of its function parameter—a mismatch Ethos missed because it only typechecks program clauses at application time. All 22 modules pass `lambdapi check` with full subject-reduction verification; the entire CPC-MINI signature is translated in ~ 100 ms and type-checked in ~ 160 ms—this is one-off work, independent of the proofs to be verified later.

4.2 Proof Benchmarks

To evaluate the translation on proofs, we sampled up to 100 unsatisfiable problems from SMT-LIB for each of the 14 fragments covered by CPC-MINI and obtained Eunoia proof scripts from CVC5, yielding 1,036 proofs. Each proof is translated by `eo21p` and the result is checked by `lambdapi check` with subject-reduction verification, under a 5-second timeout. Table 3 summarizes the results: 1,002 of 1,036 proofs (97%) are successfully translated and verified, with an average checking time of 0.47 s.

The seven equality and array fragments achieve a near-perfect pass rate (582 of 583), with the single failure being a timeout. The seven linear-arithmetic fragments pass 420 of 453 proofs (93%): `QF_UFIDL`, `UFLIA`, and `LIA` reach 98–100%; the remaining failures are 18 timeouts in `QF_LIA` and `QF_IDL` where large proofs exceed the 5-second limit, and 15 type-checking errors in `QF_UFLIA` caused by CPC rules whose type signatures mix `Real` and `Int`—most prominently `arith-int-geq-tighten`, which declares a `Real` parameter but is applied here in

⁵ <https://github.com/ciaran-matthew-dunne/cpc-mini>

Table 3. Benchmark results on 1,036 CVC5 proofs across 14 SMT-LIB fragments, grouped by theory. Each proof is translated by `eo21p` and the generated LambdaPi file is checked with subject-reduction verification under a 5 s timeout. Columns: number of proofs (**Tot.**), proofs passing (**Pass**), type-checking errors (**Chk.**), timeouts (**T/O**), average time for passing proofs (**Avg.**).

Fragment	Tot.	Pass	%	Chk.	T/O	Avg. (s)
<i>Equality</i>						
QF_UF	100	99	99	0	1	0.37
UF	100	100	100	0	0	0.51
<i>Arrays</i>						
QF_AX	72	72	100	0	0	0.50
QF_ALIA	33	33	100	0	0	0.86
QF_AUFLIA	100	100	100	0	0	0.33
ALIA	78	78	100	0	0	0.59
AUFLIA	100	100	100	0	0	0.40
<i>Linear integer arithmetic</i>						
QF_IDL	45	43	96	0	2	0.90
QF_LIA	56	42	75	0	14	0.32
LIA	100	99	99	0	1	0.44
QF_UFIDL	15	15	100	0	0	0.28
QF_UFLIA	95	80	84	15	0	0.41
UFIDL	42	41	98	1	0	0.49
UFLIA	100	100	100	0	0	0.45
Total	1,036	1,002	97	16	18	0.47

an integer-only context. Such mixed-arithmetic rules are excluded by CPC-MINI’s type-parameter tightening (change (ii) above); the failure is therefore a known *type coverage* gap at the **Real/Int** boundary, not a semantic mismatch with Ethos. Crucially, *zero* failures are caused by a semantic disagreement: every proof that Ethos accepts and we translate is accepted by LambdaPi up to coverage and time.

5 Conclusion and Future Work

We have presented an automatic translation from Eunoia to the $\lambda\Pi$ -calculus modulo rewriting, unlike approaches that hand-write soundness lemmas per rule [16, 28]. Translated rules are axioms, but their well-typedness and subject reduction are verified by LambdaPi; `eo21p` translates all 22 modules of CPC-MINI and verifies 97% of 1,036 proof scripts. Relative to Ethos, our pipeline is slower (0.47 s vs. <0.1 s) and supports a subset of CPC, but adds two guarantees Ethos does not—rule-signature well-typedness and subject reduction of side-condition programs—the latter surfaced upstream CPC bugs. Future work includes extending CPC-MINI to the full CPC, proving CPC rules sound directly in $\lambda\Pi$ [26], and using the typed interpretation of Section 3 as a setting for the higher-order extensions proposed for SMT-LIB 3 [7, 21].

Acknowledgements. Research reported in this document was supported by an Amazon Research Award, 2022, and by the CARMA associate team, whose funding for collaboration visits helped the progress of this work. The authors thank Haniel Barbosa, Hans-Jörg Schurr, Andrew Reynolds, and Yoni Zohar for discussions on the design of Eunoia and CPC, and on earlier iterations of this work. We also thank Frédéric Blanqui and Théo Winterhalter for their comments on the paper, and the rest of Deducteam for their support. This paper is dedicated to the memory of Gilles Dowek, who initiated this research project in particular and more generally the ecosystem of Dedukti.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

- [1] Michaël Armand et al. “A Modular Integration of SAT/SMT Solvers to Coq through Proof Witnesses”. In: *CPP*. Ed. by Jean-Pierre Jouannaud and Zhong Shao. Vol. 7086. LNCS. Springer, 2011, pp. 135–150. DOI: [10.1007/978-3-642-25379-9_12](https://doi.org/10.1007/978-3-642-25379-9_12).
- [2] Ali Assaf and Guillaume Burel. “Translating HOL to Dedukti”. In: *Fourth International Workshop on Proof eXchange for Theorem Proving*. Ed. by Cezary Kaliszyk and Andrei Paskevich. Vol. 186. EPTCS. 2015, pp. 74–88. DOI: [10.4204/EPTCS.186.8](https://doi.org/10.4204/EPTCS.186.8).
- [3] Ali Assaf et al. *Dedukti: a Logical Framework based on the $\lambda\Pi$ -Calculus Modulo Theory*. 2023. arXiv: [2311.07185 \[cs.LO\]](https://arxiv.org/abs/2311.07185). URL: <https://arxiv.org/abs/2311.07185>.
- [4] John Backes et al. *Semantic-based automated reasoning for AWS access policies using SMT*. 2018. URL: <https://www.amazon.science/publications/semantic-based-automated-reasoning-for-aws-access-policies-using-smt>.
- [5] Haniel Barbosa et al. “Language and Proofs for Higher-Order SMT (Work in Progress)”. In: *PxTP*. Vol. 262. EPTCS. 2017, pp. 15–22. DOI: [10.4204/EPTCS.262.3](https://doi.org/10.4204/EPTCS.262.3).
- [6] Michael Barnett et al. “Boogie: A Modular Reusable Verifier for Object-Oriented Programs”. In: *FMCO*. Ed. by Frank S. de Boer et al. Vol. 4111. LNCS. Springer, 2005, pp. 364–387. DOI: [10.1007/11804192_17](https://doi.org/10.1007/11804192_17).
- [7] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. “SMT-LIB 3: Bringing Higher-Order Logic to SMT”. In: ().
- [8] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. *The SMT-LIB Standard: Version 2.6*. Tech. rep. Department of Computer Science, The University of Iowa, 2017. URL: <https://smt-lib.org/papers/smt-lib-reference-v2.6-r2024-09-20.pdf>.
- [9] Clark Barrett et al. “Chapter 33: Satisfiability modulo theories”. In: *Frontiers in Artificial Intelligence and Applications* 336 (May 2021), pp. 1267–1329. DOI: [10.3233/FAIA201017](https://doi.org/10.3233/FAIA201017).

- [10] Jasmin Christian Blanchette, Sascha Böhme, and Lawrence C. Paulson. “Extending Sledgehammer with SMT Solvers”. In: *J. Autom. Reason.* 51.1 (2013), pp. 109–128. DOI: [10.1007/S10817-013-9278-5](https://doi.org/10.1007/S10817-013-9278-5).
- [11] Mathieu Boespflug and Guillaume Burel. “CoqInE: Translating the Calculus of Inductive Constructions into the $\lambda\Pi$ -calculus Modulo”. In: *Second International Workshop on Proof Exchange for Theorem Proving*. Ed. by David Pichardie and Tjark Weber. Vol. 878. CEUR Workshop Proceedings. 2012, pp. 44–50. URL: <https://ceur-ws.org/Vol-878/paper3.pdf>.
- [12] Chad E. Brown. *Notes on Semantics of and Proofs for SMT*. 2021. URL: <http://grid01.ciirc.cvut.cz/~chad/smtsempfs.pdf>.
- [13] Guillaume Burel et al. “First-Order Automated Reasoning with Theories: When Deduction Modulo Theory Meets Practice”. In: *J. Autom. Reason.* 64.6 (2020), pp. 1001–1050. DOI: [10.1007/s10817-019-09533-z](https://doi.org/10.1007/s10817-019-09533-z).
- [14] Guillaume Bury. “Integrating rewriting, tableau and superposition into SMT”. PhD thesis. Université Sorbonne Paris Cité, Jan. 2019. URL: <https://theses.hal.science/tel-02612985>.
- [15] Adrien Champion et al. “The Kind 2 Model Checker”. In: *CAV*. Ed. by Swarat Chaudhuri and Azadeh Farzan. Vol. 9780. LNCS. Springer, 2016, pp. 510–517. DOI: [10.1007/978-3-319-41540-6_29](https://doi.org/10.1007/978-3-319-41540-6_29).
- [16] Alessio Coltellacci, Gilles Dowek, and Stephan Merz. “Reconstruction of SMT Proofs with Lambdapi”. In: *International Workshop on Satisfiability Modulo Theories*. Vol. 3725. CEUR Workshop Proceedings. 2024, pp. 13–23. URL: <https://ceur-ws.org/Vol-3725/paper8.pdf>.
- [17] Alessio Coltellacci and Stephan Merz. “Checking Linear Integer Arithmetic Proofs in Lambdapi”. In: *FroCoS*. Vol. 15979. LNCS. Springer, 2025. DOI: [10.1007/978-3-032-04167-8_20](https://doi.org/10.1007/978-3-032-04167-8_20).
- [18] Denis Cousineau and Gilles Dowek. “Embedding Pure Type Systems in the lambda-Pi-calculus modulo”. In: *TLCA*. Ed. by Simona Ronchi Della Rocca. Vol. 4583. LNCS. Springer, 2007, pp. 102–117. DOI: [10.1007/978-3-540-73228-0_9](https://doi.org/10.1007/978-3-540-73228-0_9).
- [19] *Ethos User Manual*. URL: https://github.com/cvc5/ethos/blob/main/user_manual.md (visited on 08/12/2024).
- [20] Gabriel Hondet and Frédéric Blanqui. “The New Rewriting Engine of Dedukti”. In: *FSCD*. 167. June 2020, p. 16. DOI: [10.4230/LIPIcs.FSCD.2020.35](https://doi.org/10.4230/LIPIcs.FSCD.2020.35).
- [21] The SMT-LIB Initiative. *SMT-LIB Verion 3.0 - A Preliminary Proposal*. Dec. 2021. URL: <https://smt-lib.org/version3.shtml>.
- [22] *isabelle_dedukti: Isabelle component exporting Isabelle proofs to Dedukti*. URL: https://github.com/Deducteam/isabelle_dedukti.
- [23] Anja Petković Komel, Michael Rawson, and Martin Suda. *Case Study: Verified Vampire Proofs in the LambdaPi-calculus Modulo*. 2025. arXiv: [2503.15541](https://arxiv.org/abs/2503.15541) [cs.LO]. URL: <https://arxiv.org/abs/2503.15541>.
- [24] Nikolai Kosmatov and Julien Signoles. “Frama-C, A Collaborative Framework for C Code Verification: Tutorial Synopsis”. In: *RV*. Ed. by Yliès

- Falcone and César Sánchez. Vol. 10012. LNCS. Springer, 2016, pp. 92–115. DOI: [10.1007/978-3-319-46982-9_7](https://doi.org/10.1007/978-3-319-46982-9_7).
- [25] *Krajono: A Matita to Dedukti translator*. URL: <https://deducteam.gitlabpages.inria.fr/krajono/>.
 - [26] Hanna Lachnitt et al. “IsaRare: Automatic Verification of SMT Rewrites in Isabelle/HOL”. In: *TACAS*. Vol. 14572. LNCS. Springer, 2024, pp. 311–330. DOI: [10.1007/978-3-031-57246-3_17](https://doi.org/10.1007/978-3-031-57246-3_17).
 - [27] Cristian Mattarei et al. “CoSA: Integrated Verification for Agile Hardware Design”. In: *FMCAD*. IEEE, 2018. DOI: [10.23919/FMCAD.2018.8603014](https://doi.org/10.23919/FMCAD.2018.8603014).
 - [28] Abdalrhman Mohamed et al. “lean-smt: An SMT Tactic for Discharging Proof Goals in Lean”. In: *CAV*. Vol. 15893. LNCS. Springer, 2025, pp. 196–220. DOI: [10.1007/978-3-031-98682-6_11](https://doi.org/10.1007/978-3-031-98682-6_11).
 - [29] François Pottier and Yann Régis-Gianas. “Menhir Reference Manual”. In: (2024). Version 20240715. URL: <http://gallium.inria.fr/~fpottier/menhir/>.
 - [30] Léa Riant. “Debugging Support in Atelier B”. In: *SEFM 2022 Collocated Workshops Revised Selected Papers*. Ed. by Paolo Masci et al. Springer, 2023, pp. 148–155. DOI: [10.1007/978-3-031-26236-4_12](https://doi.org/10.1007/978-3-031-26236-4_12).
 - [31] Neha Rungta. “A Billion SMT Queries a Day”. In: *Computer Aided Verification (CAV)*. Vol. 13371. Lecture Notes in Computer Science. Springer, 2022, pp. 3–18. DOI: [10.1007/978-3-031-13185-1_1](https://doi.org/10.1007/978-3-031-13185-1_1).
 - [32] Hans-Jörg Schurr et al. “Alethe: Towards a Generic SMT Proof Format (Extended Abstract)”. In: *Electronic Proceedings in Theoretical Computer Science* 336 (2021), pp. 49–54. DOI: [10.4204/EPTCS.336.6](https://doi.org/10.4204/EPTCS.336.6).
 - [33] Aaron Stump et al. “SMT proof checking using a logical framework”. In: *Formal Methods Syst. Des.* 42.1 (2013), pp. 91–118. DOI: [10.1007/S10703-012-0163-3](https://doi.org/10.1007/S10703-012-0163-3).
 - [34] Geoff Sutcliffe, Frédéric Blanqui, and Guillaume Burel. “Proof Verification with GDV and LambdaPi - It’s a Matter of Trust”. In: *FLAIRS*. Ed. by Douglas A. Talbert and Ismaïl Biskri. Florida Online Journals, 2025. DOI: [10.32473/FLAIRS.38.1.138642](https://doi.org/10.32473/FLAIRS.38.1.138642).
 - [35] Rishikesh Vaishnav. “Lean4Less: Eliminating Definitional Equalities from Lean via an Extensional-to-Intensional Translation”. In: *ICTAC 2025*. LNCS. Accepted, to appear. Springer, 2025. URL: <https://rish987.github.io/files/lean4less.pdf>.